

Recursion and the Call Stack

From factorial to recursive DFS on a maze

CSCI 235 – Data Structures

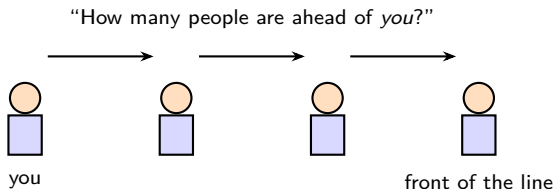
Hunter College, Fall 2026

Class 11 Tuesday, October 6, 2026

Goals for Today

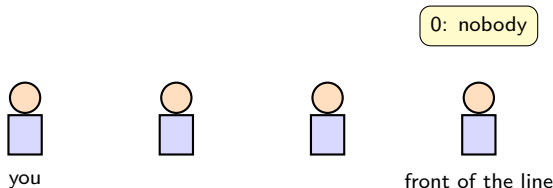
- What is recursion, and why use it?
- The recipe: base case, recursive case, combine
- Examples
 - calculate a factorial
 - print a string in reverse order
 - add up the integers in an array
- What the computer does: the **call stack**
- A first real use: **recursive DFS** on a maze (Project 2, Task A)

Warm-Up: How Many People Are Ahead of You?



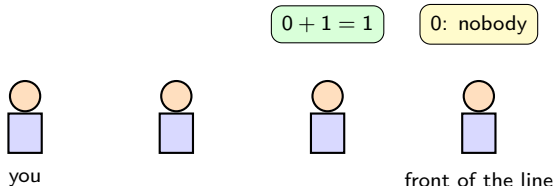
You cannot see the whole line. So you ask the person ahead the *same question*, and so does everyone else.

Warm-Up: How Many People Are Ahead of You?



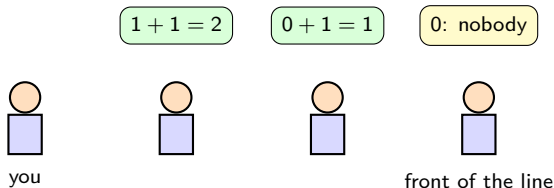
The person at the front has nobody ahead: they know the answer directly. This is the **base case**.

Warm-Up: How Many People Are Ahead of You?



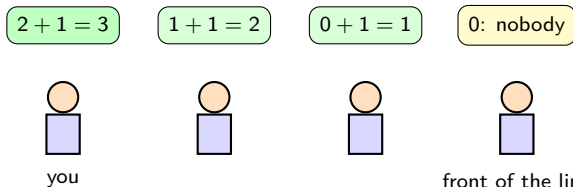
Each person takes the answer from the person ahead and adds 1.

Warm-Up: How Many People Are Ahead of You?



Answers travel back down the line.

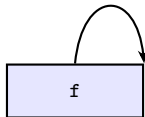
Warm-Up: How Many People Are Ahead of You?



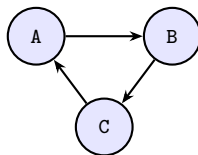
You get 3, without ever counting the line yourself. That is recursion.

What Is Recursion?

A function that **calls itself**, directly or indirectly, while making *progress toward a base case*: a smaller number, a shorter list, or fewer unvisited cells.



direct: f calls f



indirect: A calls B, B calls C,
C calls A

Today we use direct recursion.

Why Recursion?

- A new **mode of thinking**, and a powerful programming tool.
- Recursion appears in many algorithmic patterns, including **divide and conquer**:
 - **divide** the problem into smaller pieces,
 - **conquer** the pieces (the smallest are solved directly),
 - **combine** the answers.
- Some problems lead naturally to recursive exploration: from one maze cell, explore its neighbors with the same procedure. Trees are recursively structured data: every subtree is another tree (BST 11/19, tries 11/24).
- Coming up: DFS and BFS (P2), backtracking (P3), merge sort and quicksort.

Solving a Problem with Recursion

The recipe

- 1 **Base case:** solve the smallest problem *directly*.
- 2 **Recursive case:** move one step closer to the base case (for example, a smaller input) and call the function again.
- 3 **Combine:** after the recursive call, use its result (or finish the work still needed) to solve the original problem.

The leap of faith

Assume the recursive call already works for every case closer to a base case. Your job: handle the base case, and build each case from the one closer to it.

Key ideas

Each call must make **progress toward a base case**. A **base case** gives a direct answer. Recursion must **stop**.

Example 1: Factorial (the Non-Recursive Definition)

For a positive integer n ,

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 1, \quad 0! = 1.$$

- $3! = 3 \times 2 \times 1 = 6$, $5! = 120$.
- What does $n!$ count? The number of ways to **line up** n different objects. Five students can stand in a row in $5! = 120$ ways.

Factorial: the Recursive Definition

Since

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 1$$

$$(n-1)! = (n-1) \times (n-2) \times \cdots \times 1,$$

we can write $n!$ in terms of $(n-1)!$:

$$n! = n \times (n-1)!$$

Two parts of a recursive definition

Base case (a direct answer): $0! = 1$.

Recursive case (in terms of a smaller problem): for $n \geq 1$,
 $n! = n \times (n-1)!$.

Factorial in C++

```
1 // long, since factorial grows fast.
2 // Assume n >= 0 and n! fits in long.
3 long factorial(int n) {
4     if (n == 0){
5         return 1;
6     }
7     else {
8         return n * factorial(n-1);
9     }
10 }
```

- Lines 4–6: the **base case**, `n == 0`.
- Line 8: the **recursive case**. The argument `n-1` is *smaller*, and `n *` is the **combine** step.

Tracing factorial(3)

```
factorial(3) = 3 * factorial(2)
```

Going down: each call waits for a smaller call.

Tracing factorial(3)

```
factorial(3) = 3 * factorial(2)
```

```
factorial(2) = 2 * factorial(1)
```

Going down: each call waits for a smaller call.

Tracing factorial(3)

`factorial(3) = 3 * factorial(2)`

`factorial(2) = 2 * factorial(1)`

`factorial(1) = 1 * factorial(0)`

Going down: each call waits for a smaller call.

Tracing factorial(3)

`factorial(3) = 3 * factorial(2)`

`factorial(2) = 2 * factorial(1)`

`factorial(1) = 1 * factorial(0)`

`factorial(0) = 1 (base case)`

The base case answers directly: `factorial(0)` is 1.

Tracing factorial(3)

`factorial(3) = 3 * factorial(2)`

`factorial(2) = 2 * factorial(1)`

`factorial(1) = 1 * factorial(0)`

`factorial(0) = 1 (base case)`

Now come back up. Each waiting call receives an answer.

Tracing factorial(3)

`factorial(3) = 3 * factorial(2)`

`factorial(2) = 2 * factorial(1)`

`factorial(1) = 1 * factorial(0) = 1 * 1 = 1`

`factorial(0) = 1 (base case)`

`factorial(1)` gets 1 and returns $1 \times 1 = 1$.

Tracing factorial(3)

`factorial(3) = 3 * factorial(2)`

`factorial(2) = 2 * factorial(1) = 2 * 1 = 2`

`factorial(1) = 1 * factorial(0) = 1 * 1 = 1`

`factorial(0) = 1 (base case)`

`factorial(2)` gets 1 and returns $2 \times 1 = 2$.

Tracing factorial(3)

`factorial(3) = 3 * factorial(2) = 3 * 2 = 6`

`factorial(2) = 2 * factorial(1) = 2 * 1 = 2`

`factorial(1) = 1 * factorial(0) = 1 * 1 = 1`

`factorial(0) = 1 (base case)`

`factorial(3)` gets 2 and returns $3 \times 2 = 6$.

Watch It Run

The same function with a print statement added at the start and the end of each call:

```
factorial(3) called
  factorial(2) called
    factorial(1) called
      factorial(0) called
      factorial(0) returns 1
    factorial(1) returns 1
  factorial(2) returns 2
factorial(3) returns 6
```

- Calls **open** in order 3, 2, 1, 0.
- They **finish** in the opposite order: 0, 1, 2, 3. The last call made is the first to finish.

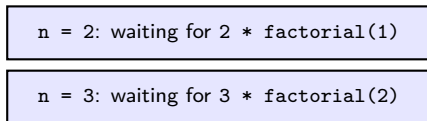
What the Computer Does: Stack Frames

`factorial(3)` is called.
It gets its own **frame**
holding its own `n`.

`n = 3: waiting for 3 * factorial(2)`

oldest call (bottom)

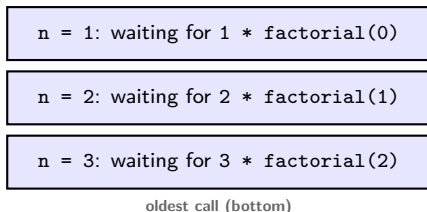
What the Computer Does: Stack Frames



oldest call (bottom)

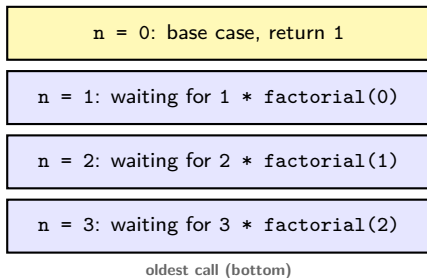
It calls `factorial(2)`:
a *new* frame goes on
top. The old one waits.

What the Computer Does: Stack Frames



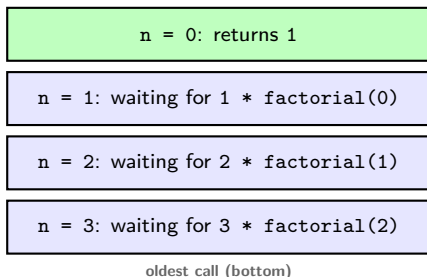
Another call, another frame. Each frame has its *own* n .

What the Computer Does: Stack Frames



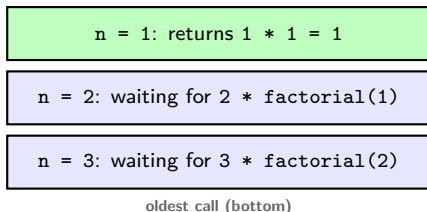
$n == 0$: the base case.
No more calls.

What the Computer Does: Stack Frames



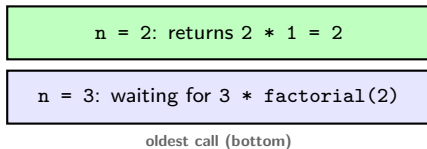
The top frame returns 1
and is **removed**.

What the Computer Does: Stack Frames



The frame beneath it resumes and computes its answer.

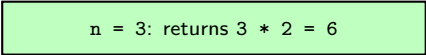
What the Computer Does: Stack Frames



Again: return, remove,
resume.

What the Computer Does: Stack Frames

This final factorial frame returns 6 and is removed; execution resumes in the caller.



`n = 3: returns 3 * 2 = 6`

oldest call (bottom)

Recursion Uses a Stack You Did Not Write

- Every function call (recursive or not) gets a **stack frame**: its parameters, local variables, and where to resume.
- If A calls B, which calls C: C runs, then returns to B, then B returns to A. The **last** call is the **first** to finish.
- That is **last in, first out**: exactly the **stack ADT** from Class 9. A call is a push; a return is a pop.
- A recursive call is just an ordinary call that happens to be to the same function. Each frame keeps its own copy of n , so the calls do not interfere.

Remember

Recursion needs a stack, but the computer keeps it for you. Later we will build the same algorithm with our own explicit stack.

When Recursion Goes Wrong

```
long bad(int n) {  
    return n * bad(n - 1); // no base case!  
}
```

- `bad(3)` calls `bad(2)`, then `bad(1)`, `bad(0)`, `bad(-1)`, ... forever: no call ever returns.
- Every call adds a frame, and the call stack has **limited space**. When it runs out, the program crashes (on our test machine: Segmentation fault). This is a **stack overflow**.

Two checks for every recursive function

1. Is there a base case that stops the calls?
2. Does every recursive call make *progress* toward the base case, so it must eventually reach it?

Practice 1: Print a String Backward

Given "abc de", print "ed cba".

- 1 What is the **shortest** string? How do you print it?
- 2 For a longer string, how do you divide it into a smaller case?
- 3 How do you **combine** the answers?

Practice 1: Print a String Backward

Given "abc de", print "ed cba".

- 1 What is the **shortest** string? How do you print it?
- 2 For a longer string, how do you divide it into a smaller case?
- 3 How do you **combine** the answers?

Think

Idea: to print *s* backward, first print the rest of the string (everything after the first character) backward, *then* print the first character.

Print a String Backward: Version A

```
void printReverse(std::string s) {  
    if (s.empty()) {           // base case:  
        return;               // nothing left  
    }  
    printReverse(s.substr(1)); // smaller: rest of s  
    std::cout << s[0];       // combine: then s[0]  
}
```

`s.substr(1)` is `s` without its first character. The call `printReverse("abc")` prints `cba`:

<code>printReverse("abc")</code>	→ calls <code>("bc")</code> , then prints <code>a</code>	(last)
<code>printReverse("bc")</code>	→ calls <code>("c")</code> , then prints <code>b</code>	
<code>printReverse("c")</code>	→ calls <code>("")</code> , then prints <code>c</code>	(first)
<code>printReverse("")</code>	→ base case: prints nothing	

Print a String Backward: Version B

```
void printReverse(const std::string& s,
                  std::string::size_type i) {
    if (i == s.length()) { // base case:
        return;           // nothing left
    }
    printReverse(s, i + 1); // smaller: rest of s
    std::cout << s[i];     // combine: then s[i]
}
```

i is the position of the first character not yet handled (`std::string::size_type` is the type strings use for positions). Call it as `printReverse(s, 0)`. For "abc": the calls are $i = 0, 1, 2, 3$; $i = 3$ is the base case; on the way back it prints c, b, a.

Which is more efficient?

Version A builds a new, shorter string in every call (about $n - 1, n - 2, \dots$ characters copied). Version B copies nothing: s is passed by reference and only i changes. Same idea, less work.

Practice 2: Sum of an Array

Find the sum of the integers in an array of n integers.

- 1 Smallest array? (**base case**)
- 2 How can the sum of n numbers use the sum of $n - 1$ numbers?

Practice 2: Sum of an Array

Find the sum of the integers in an array of n integers.

- 1 Smallest array? (**base case**)
- 2 How can the sum of n numbers use the sum of $n - 1$ numbers?

Think

The sum of the first n numbers is the sum of the first $n - 1$ numbers, plus the last one.

Sum of an Array: Solution

```
int sumArray(const int a[], int n) {  
    if (n == 0) {                // base case:  
        return 0;                // empty sum  
    }  
    return a[n - 1]              // last element  
        + sumArray(a, n - 1);    // + sum of the rest  
}
```

sumArray(a, 3) for a = {2, 5, 3}:

$$3 + (5 + (2 + 0)) = 10$$

- Sizes shrink: $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$. The empty array is the base case.
- Same shape as factorial: a base case, a smaller call, and a combine step (here +).

Recursion or a Loop?

- Factorial, reversal and sum can all be written with a for loop, usually more cheaply. A recursive call uses a stack frame; a loop does not.
- Recursion is worth it when the problem is **naturally solved by the same procedure applied to a part of it**: a tree (each subtree is a tree), a maze search (“explore a neighbor the same way”).
- The cost to remember: depth. If n recursive calls are active at the same time, there are n stack frames, so a very deep recursion can overflow the stack.

Our next example

Searching a maze with **depth-first search**. Recursive DFS is natural; an iterative DFS uses an explicit stack.

A Small DFS Example, Step by Step

	0	1	2
0	S		
1			
2			E

A 3×3 maze. Black cells are walls. Start $(0,0)$ = row 0, column 0; end $(2,2)$.

Neighbors are tried in the order **left, right, up, down**, as in Project 2.

The following slides show individual calls and backtracks; the final slide summarizes the successful unwinding.

Watch three things: the **visited** cells, the **call stack**, and when the **answer stack** gets filled. Stacks are drawn with the **top first**.

Orange: current call. Blue: visited, call still active. Gray: dead end, already returned. Green: final route.

Small DFS Example: Step 1 of 10

	0	1	2
0	S		
1			
2			E

Call stack

helper(0,0)

Answer stack

(empty)

Start: call `helper(0,0)`. Mark it visited. Neighbors: left outside, right **open**. Go there next.

Small DFS Example: Step 2 of 10

	0	1	2
0	S		
1			
2			E

Call stack

helper(0,1)
helper(0,0)

Answer stack

(empty)

Move right: call `helper(0,1)`. Mark it visited. Neighbors: left visited, right **open**. Go there next.

Small DFS Example: Step 3 of 10

	0	1	2
0	S		
1			
2			E

Call stack

helper(0,2)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move right: call `helper(0,2)`. Mark it visited. Neighbors: left visited, right outside, up outside, down wall. **Dead end**: return.

Small DFS Example: Step 4 of 10

	0	1	2
0	S		
1			
2			E

Call stack

helper(0,1)
helper(0,0)

Answer stack

(empty)

Backtrack: the call returned empty-handed and left the stack, so we are back in `helper(0,1)`. Remaining neighbors: up outside, down **open**. Go there next.

Small DFS Example: Step 5 of 10

	0	1	2
0	S		
1			
2			E

Call stack

helper(1,1)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move down: call `helper(1,1)`. Mark it visited. Neighbors: left wall, right wall, up visited, down **open**. Go there next.

Small DFS Example: Step 6 of 10

	0	1	2
0	S		
1			
2			E

Call stack

helper(2,1)
helper(1,1)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move down: call `helper(2,1)`. Mark it visited. Neighbors: left **open**. Go there next.

Small DFS Example: Step 7 of 10

	0	1	2
0	S		
1			
2			E

Call stack

helper(2,0)
helper(2,1)
helper(1,1)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move left: call `helper(2,0)`. Mark it visited. Neighbors: left outside, right visited, up wall, down outside. **Dead end:** return.

Small DFS Example: Step 8 of 10

	0	1	2
0	S		
1			
2			E

Call stack

helper(2,1)
helper(1,1)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Backtrack: the call returned empty-handed and left the stack, so we are back in `helper(2,1)`. Remaining neighbors: right **open**. Go there next.

Small DFS Example: Step 9 of 10

	0	1	2
0	S		
1			
2			E

Call stack

helper(2,2)
helper(2,1)
helper(1,1)
helper(0,1)
helper(0,0)

Answer stack

(2,2)

Move right: call `helper(2,2)`. Mark it visited. It is the **end** (base case): push it on the answer stack and return success.

Small DFS Example: Step 10 of 10

	0	1	2
0	S		
1			
2			E

Call stack

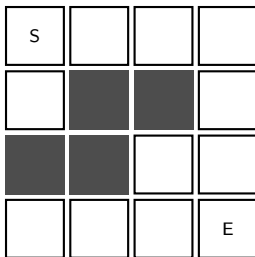
(no calls left)

Answer stack

(0,0)
(0,1)
(1,1)
(2,1)
(2,2)

Success unwinds the call stack. Each frame sees a non-empty answer stack, pushes its own cell and returns. Popping the answer stack gives the route: $(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2)$. Cells $(0,2)$ and $(2,0)$ were dead ends.

Recursive DFS: Finding a Path in a Maze



A larger maze, from the Project 2 handout. Black cells are walls. Start (0,0), end (3,3). Neighbors are tried in the order **left, right, up, down**. Numbers show the order of the calls.

Is there a path from S to E?
Which one will DFS find?

Recursive DFS: Finding a Path in a Maze

S/1	2	3	4
			E

A larger maze, from the Project 2 handout. Black cells are walls. Start $(0,0)$, end $(3,3)$. Neighbors are tried in the order **left, right, up, down**. Numbers show the order of the calls.

First four calls: along the top row. Left is outside or already visited, so each call tries *right*.

Recursive DFS: Finding a Path in a Maze

S/1	2	3	4
			5
		7	6
		8	E

A larger maze, from the Project 2 handout. Black cells are walls. Start (0,0), end (3,3). Neighbors are tried in the order **left, right, up, down**. Numbers show the order of the calls.

Down the right side to (2,3), then *left* (before down) to (2,2) and (3,2): 8 calls.

Recursive DFS: Finding a Path in a Maze

S/1	2	3	4
			5
		7	6
10	9	8	E

A larger maze, from the Project 2 handout. Black cells are walls. Start (0,0), end (3,3). Neighbors are tried in the order **left, right, up, down**. Numbers show the order of the calls.

From (3,2), *left* comes first: calls 9 and 10. Dead end: both return empty-handed (backtracking).

Recursive DFS: Finding a Path in a Maze

S/1	2	3	4
			5
		7	6
10	9	8	E/11

A larger maze, from the Project 2 handout. Black cells are walls. Start (0,0), end (3,3). Neighbors are tried in the order **left, right, up, down**. Numbers show the order of the calls.

Back at (3,2), the next neighbor, *right*, is the end. The route (green, 9 cells) is built as the calls return. BFS will find a shorter one (7 cells) next class.

The Recursive Idea

The question each call asks

“Starting from this cell, is there a path to the end?”

- **Mark** the cell visited. **Base case:** if it is the end, we succeeded.
- **Recursive case:** for each open, unvisited neighbor (left, right, up, down), ask the *same question* from it.
- **Leap of faith:** if a neighbor's call succeeds, so does this call. If *every* neighbor fails, it is a **dead end**: the call returns and its caller tries the next neighbor. That return is the **backtrack**.
- The visited table is **shared** and cells are **never un-marked**. Each call marks one more cell, so unvisited cells shrink: that is the progress.

Recursive DFS: Pseudocode

```
helper(cell):
    mark cell visited
    if cell is the end:
        push cell onto the answer stack
        return // success
    for each neighbor, in order
        left, right, up, down:
            if neighbor is open and
                not visited:
                helper(neighbor) // go forward
                if the answer stack is
                    not empty: // success came back
                        push cell; return
    // every neighbor failed: return to
    // the caller, which tries its next
    // choice (the backtrack)
```

- The answer is built as the recursion **unwinds**: end first, start last (on top).
- Project 2, Task A: write this in C++.

DFS and the Call Stack

- Each call to `helper` is a stack frame. The frames on the stack are the **current search route** from the start to the current cell (the ball of string). A dead end leaves it when it returns, so it is *not* the final answer.
- Stepping into a neighbor = **push** a frame. A dead end returns empty-handed = **pop**, and the caller continues with its next neighbor. That is backtracking.
- When the end is reached, every frame below it sees a non-empty answer stack and pushes its own cell: the stack **unwinds along the path**.

Why the visited table?

Mazes have loops. Without marking cells, the search can recurse forever: a recursion that makes no progress.

Summary

- In a recursive function, find the **base case** first.
- **Assume** the smaller case works; use it to finish the current case.
- Every recursive call must make **progress** toward a base case; otherwise the stack overflows and the program crashes.
- Each call has its own **stack frame**; on return, the most recently called function that has not yet returned finishes first (last in, first out).
- **Recursive DFS** tries the neighbors in a fixed order, trusts the recursive call, and **backtracks** when every neighbor fails.

Quick Check

- 1 What does `factorial(4)` return? How many frames are on the stack at the deepest point?
- 2 What is wrong with `int f(int n) { return n + f(n - 1); }`?
- 3 In `sumArray`, what does `sumArray(a, 0)` return, and why is that the right answer?
- 4 In recursive DFS, what happens in a call when every neighbor fails to find the end?

Next class (Thu 10/8): BFS and DFS vs. BFS. Ex 5 is due.

Quick Check

- 1 What does `factorial(4)` return? How many frames are on the stack at the deepest point?
- 2 What is wrong with `int f(int n) { return n + f(n - 1); }`?
- 3 In `sumArray`, what does `sumArray(a, 0)` return, and why is that the right answer?
- 4 In recursive DFS, what happens in a call when every neighbor fails to find the end?

Answers

1. 24; five frames ($n = 4, 3, 2, 1, 0$).
2. No base case, so it never stops: stack overflow.
3. 0: the sum of no numbers is 0, so adding `a[n-1]` to it works.
4. It returns without adding anything to the answer stack: a dead end, so the caller tries its next neighbor.

Next class (Thu 10/8): BFS and DFS vs. BFS. Ex 5 is due.